

AKASH HADAGALI PERSETTI

Bloomington, IN | 812-837-7651 | hadagalipersettiakash@gmail.com
linkedin.com/in/akash-hp | github.com/akashpersetti | akashpersetti.com

SUMMARY

AI engineer building agentic systems that stay reliable in production, and the evaluation infrastructure that proves whether they do. Ship LLM pipelines behind deterministic gates – schema validation, real compiler checks, human approval, hard cost caps – and measure them rather than assuming: an 8-point schema-validity gap between providers, a retry loop that added 3x latency for zero pass-rate gain, a silent 15-point retrieval regression caught by my own harness. Full-stack across LangGraph, FastAPI, Next.js, and serverless AWS provisioned with Terraform and OIDC.

TECHNICAL SKILLS

Languages & Tools: Python, TypeScript, JavaScript, SQL, C++, Docker, Terraform (IaC), GitHub Actions CI/CD with OIDC, pytest

Agentic AI, LLM Engineering & Evaluation: LangGraph, LangChain, OpenAI SDK, LiteLLM, Model Context Protocol (published to PyPI), RAG, function calling, structured outputs, Pydantic validation, prompt caching, multi-agent workflows, human-in-the-loop gating, guardrails and cost caps; schema-validity and retry accounting, recall@k, nDCG@k, MRR, LLM-as-judge with variance tracking, Wilson and binomial confidence intervals, latency/cost benchmarking, regression detection in CI

Cloud, Backend & Data: AWS (Lambda, Bedrock, Fargate, Step Functions, API Gateway, DynamoDB, SQS, S3, CloudFront, EFS, ECR, SES), serverless and event-driven architecture, FastAPI, Next.js, React, Node.js, PostgreSQL, MySQL, Supabase, vector databases, embeddings, Hugging Face, scikit-learn, MediaPipe

EXPERIENCE

Software Engineer Intern

Humaximus

Sep 2026 – Present

Remote

- Sole engineer, working directly with the CEO, rebuilding a healthcare and community-coordination platform for a Dallas–Fort Worth pilot where partner organizations coordinate care for one participant across referrals, cases, tasks, and consent-governed data sharing. Four portals (staff, partner, patient, helper) on a single Next.js 16 / React 19 / TypeScript app with server actions over Postgres.
- Built the AI layer as a single governed gateway — the only code permitted to reach a model provider — supporting Gemini 2.5 Flash, Kimi, and Claude Sonnet, switchable from an admin setting so a provider change is configuration rather than a code change.
- Designed the agents to be recommend-only by construction: episode briefings, stalled-work detection, and recovery-plan drafts operate on a permission-filtered view of the participant, cannot write data, and emit an audit record on every call. In a consent-governed health context, an agent that cannot write is the difference between a suggestion and an incident.
- Authored the AWS target architecture (ECS Fargate for compute, Bedrock for models, S3 for files) with a service-by-service cost model: roughly \$300–450/month consolidated on AWS against \$800–870/month retaining Supabase alongside it. Selected Bedrock primarily to cut vendor HIPAA business associate agreements from 5 to 2, and held the Aurora Serverless v2 database move pending confirmation that private VPC connectivity is available.
- Led the database cutover and ported the prototype into the production application; the normalized 65-table `hx.*` schema now carries row-level security on 34 tables, with the remaining tenant tables tracked as outstanding work ahead of the pilot.

AI Engineer (Remote)

MyEdMaster LLC

Jun 2026 – Present

Leesburg, VA

- Own the LLM integration layer of an adaptive learning platform that assesses a learner’s existing knowledge of a certification and generates coursework covering only the gaps; 111 of 140 commits on the main line across a five-person team, backed by 72 tests.
- Built a `structured.completion` client over LiteLLM with validation retry and a JSON-mode fallback, and enforced two-layer Pydantic schemas across all four pipeline stages so provider-shaped responses never leak into domain types and a prompt change does not become a schema migration.
- Moved question-type allocation out of the prompt into deterministic Python with post-validation retry, because models distribute counts subtly wrong in ways the output still looks correct for – a miscounted domain weighting reaches someone studying for a real certification exam.
- Diagnosed a persistent HTTP 402 “insufficient credits” error as burst-rate throttling rather than billing, by checking the balance against the provider’s own API and reproducing it outside the application with raw HTTP calls; LiteLLM’s built-in retry fires with no delay and re-enters the same rate window, so I replaced it with tenacity exponential backoff (10s floor, measured) and disabled the native retries so the loops cannot compound.

- Traced a 500-ing course-list endpoint to `select_related` pulling 10–26MB JSON payload columns into a sorted query that needed five scalar fields, overflowing the MySQL sort buffer on 17 rows; rewrote it to an explicit `.values()` projection rather than raising the buffer, which would have hidden the defect until the data grew.
- Defined the answer-key stripping contract after expanding assessments to four question types introduced three new answer-revealing fields: all four stripped before reaching the client, and grading loads the server-side assessment rather than trusting a client-returned one. Implemented by a teammate twelve days later.

Founding Engineer

Jun 2026 – Jul 2026

Laxora.ai

Remote

- Co-founded a HIPAA-compliant AI scheduling assistant for dental and physician offices; owned the AWS architecture and cost model while my co-founder owned Terraform provisioning and DNS. Reached a working pre-launch POC with the architecture and compliance design settled.
- Argued for Bedrock over direct provider APIs on compliance rather than model grounds – we were already signing an AWS BAA for RDS and S3, and any third-party LLM would need its own BAA negotiated separately. Adopted, at the known cost of giving up provider optionality I had measured to be worth an 8-point schema-validity gap elsewhere.
- Caught that CloudTrail logs AWS API actions but not application-layer access to patient data, so the audit trail everyone assumed covered PHI access did not; specified conversation-level audit logging separately. Found four days into the project after learning HIPAA and BAA requirements from a standing start.
- Revised the infrastructure cost model service by service, correcting Lambda cost as duration-bound (1–2s blocked per turn on the model call) rather than request-bound and flagging that “automated failover” was untrue as written without Multi-AZ; POC estimate moved from a blended \$148 to roughly \$115/month. Specified prompt caching on the system prompt and per-practice config, projected from published token pricing to cut roughly 90% of the dominant input cost, designed to survive a provider pivot by mapping onto a Bedrock `InvokeModel` call (projected only — the product never carried production traffic). Built and shipped the public marketing site in Next.js static export in a single day.

Machine Learning Intern (Remote)

Jun 2025 – Aug 2025

MyEdMaster LLC

Leesburg, VA

- Built a real-time computer-vision exercise-assessment system in Python and MediaPipe, holding form-detection latency under 100ms per frame through pose-landmark extraction and joint-angle calculation, delivering sub-second corrective feedback so users self-corrected during live sessions without instructor oversight.
- Cut counting errors from 20 to 14 over 200 ground-truth reps (over 92% count accuracy across kettlebell front raises, seated leg extensions, and jack knives) by adding calibration routines and motion-consistency checks rather than a heavier model, keeping the latency budget intact; form-error events requiring coach intervention dropped from 50 to 30 on the same test set.

PROJECTS

RepoModernizer | *Autonomous Codebase Migration Agent* | LangGraph, Fargate repo-modernizer.akashpersetti.com

- Built a 5-node LangGraph state machine (ingest, install deps, plan, migrate-file loop, finalize) that plans a file-by-file migration from a natural-language goal, rewrites each file with an LLM, and gates any diff above a risk threshold behind a human-in-the-loop `interrupt()` before it touches disk.
- Computed diffs from known-good before/after content rather than trusting LLM-authored diffs, with bounded retries and Bedrock primary/fallback failover on transient errors, plus guardrails (forbidden paths, max changed lines, no deletions) and a hard per-task cost cap evaluated against estimated spend *before* each call rather than after.
- Fixed a concurrency defect where repeated approve clicks produced 6–9 duplicate commits, using a pending-interrupt check plus an atomic DynamoDB claim so exactly one resume wins; verified by inducing crashes mid-migration and confirming correct file state and no duplicate commits on resume.
- Root-caused a `git apply` rejection affecting roughly a third of files (10 of 31, consistent across two batches) to diff generation force-padding a trailing newline onto both sides, which misrepresents files that genuinely lack one – invisible to Python fixtures, fatal on a real JS repo. Fixed upstream by normalizing before diffing rather than hand-rolling the `\ No newline at end of file` markers `difflib` does not support.
- Ran 16 real migrations at \$0.0023 and 4.8 minutes average per run on a deliberately NAT-free stack (API Gateway, Lambda via Mangum, SQS, one-shot Fargate, EFS-mounted workspace, DynamoDB checkpointing) after pricing showed a NAT gateway would cost more per month than the bursty workloads it served; provisioned via Terraform and GitHub Actions OIDC, 96 passing tests, no long-lived AWS credentials.

TerraformAgent | *Multi-Agent IaC Generator* | LangGraph, FastAPI, Step Functions terraform-agent.akashpersetti.com

- Built a 6-node LangGraph pipeline (orchestrator, researcher, parallel domain subagents, aggregator, reviewer, evaluator) that turns a natural-language infrastructure request into validated Terraform, with a conditional edge that reruns only the domains a review flags, up to a capped number of targeted retries.
- Gated output on a real compiler rather than a model’s opinion of its own work: the evaluator writes generated code to disk and runs `terraform fmt` and `terraform validate` against a provider cache baked into the Docker image, reaching full offline validation by copying the read-only cache into a writable path once per execution environment.

- Instrumented the targeted-retry loop and measured it across 10 real prompts in both modes: zero pass-rate improvement (the same 4 of 10 passed) at 46s to 145s average wall time. Root-caused to a severity classifier that treats any wildcard resource as blocking – firing on 10 of 10 tasks so the loop always engages and always maxes out – plus cross-domain reference errors that single-domain regeneration cannot fix by construction.
- Hardened review with a multi-LLM security fan-out (reusing the mcp-second-opinion pattern) querying OpenAI and Anthropic in parallel and blocking on any veto, degrading to single-LLM review on provider outage. Documented its real limits: it caught an IAM policy granting `kms:Encrypt` on `Resource: "*"` precisely and then failed to repair it across three regenerations, and it cannot distinguish an intentionally public S3 bucket for static hosting from an accidentally public one.
- Deployed the containerized service on Lambda and Step Functions with JWT authentication, DynamoDB state, SQS failure handling, and Secrets Manager; provisioned through Terraform and GitHub Actions OIDC, with 94 passing tests and no long-lived AWS credentials.

EvalBench | *Multi-Provider LLM Eval Platform* | FastAPI, LiteLLM, Next.js, Terraform evalbench.akashpersetti.com

- Architected a pluggable-suite eval harness where every benchmark emits one shared `MetricRecord` contract persisted to a single JSON-column table, so adding a suite is a three-file change that never touches the runner, store, or dashboard core.
- Shipped 3 benchmark suites on that harness: structured-output reliability (schema-valid rate, retries-to-valid, retry-only cost accounting), latency and cost with pairwise judge scoring, and RAG with recall@k, nDCG@k, and faithfulness across swappable chunking strategies. Hybrid scoring by design – deterministic checks wherever a right answer exists, LLM-as-judge only where none does.
- Ran 30 evaluation runs across 75 tasks and 7 model/config variants (330 result rows), surfacing an 8-point first-attempt schema-validity gap between providers on structured output, and a 15x embedding-cost difference between chunking strategies on the same corpus (209K vs 1.2M prompt tokens, \$0.018 vs \$0.263).
- Disaggregated a 48% row-level error rate into two distinct causes rather than treating it as one: rate limiting, real but historical (44% of July rows, 10% by August after mitigation), and a new judge response-parsing failure (0% to 43%). Ruled out the obvious explanation by checking that judge variance was near zero – the judge agrees with itself when it succeeds, so this is a parsing and validation problem, not unreliable judgment, and needs an entirely different fix.
- Hand-implemented statistically honest aggregation: 95% Wilson intervals on proportions (chosen over the normal approximation, which misbehaves near 0 and 1 at small n), binomial order-statistic intervals on p95 latency, and non-negative quantities clamped at zero rather than bootstrapped – a deterministic change over a new resampling dependency – with per-metric sample sizes on every leaderboard cell so no aggregate renders as a bare mean. Deployed serverless on AWS (dual Lambda, API Gateway v2, CloudFront, DynamoDB, SES magic-link auth) with 262 passing tests that run against injected LiteLLM callables and temporary SQLite databases, so the full suite costs nothing in provider calls.

Twain | *AI Digital Twin + Live Eval Dashboard* | FastAPI, Next.js, AWS Bedrock, Lambda, S3 akashpersetti.com

- Built a streaming personal-website agent using Titan-embedding retrieval over a persona corpus, grounded in a profile assembled at Lambda cold-start from a LinkedIn PDF, career summary, and style guide (parsed via pypdf) and injected into a Bedrock Claude system prompt, with an escalate-to-human path via SNS and a DynamoDB admin inbox when the agent should not guess.
- Shipped a public evals dashboard scoring retrieval quality (recall@k, nDCG@k, MRR) on 35 labeled queries per push, plus LLM-judged faithfulness on live traffic via an S3-event-driven judge Lambda that adds zero latency to chat responses.
- Used that harness to catch a silent quality regression in my own system: recall@5 had slid from 0.837 to 0.682 across four profile updates I had treated as content changes not warranting re-evaluation. Evaluation now runs against the live profile rather than a stale snapshot. Also documented a known harness bias – the judge never sees `facts.json` or `summary.txt`, so faithfulness reads low for facts that are legitimately available but unretrieved.

mcp-second-opinion | *Open-Source MCP Server, PyPI* | Python, LiteLLM pypi.org/project/mcp-second-opinion

- Built and published an MIT-licensed MCP stdio server that lets any MCP-aware agent consult rival LLMs (OpenAI, Gemini, Anthropic, Grok) mid-conversation, exposing exactly two tools: query a single model, or fan out to all enabled providers in parallel and compare. 425 downloads excluding mirrors.
- Unified four provider APIs behind a single LiteLLM interface returning per-call latency, token counts, and cost; a provider with no API key disables itself and a provider that errors mid-fan-out returns its error alongside the others' answers rather than taking the call down. Ships with 49 tests and the design spec written before the implementation.
- Kept the surface deliberately small, which is what made it portable: the same parallel fan-out dropped directly into TerraformAgent as its multi-LLM security review stage.

EDUCATION

Indiana University Bloomington – M.S. Computer Science, GPA 3.8/4.0

Graduated May 2026

The National Institute of Engineering, Mysuru, India – B.Eng. Computer Science, GPA 3.7/4.0

2020 – 2024

Certifications: Proficient AI Engineer – Ed Donner / The AI Engineer track (2026) | Introduction to Machine Learning – NPTEL, IIT Madras (2023)